Field Hours

CSCE 638 Natural Language Processing

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Counts over a finite vocabulary

You choose tokens before any model. Split on spaces and lowercase, so the word with a capital and the same word without one share a type. If a type occurs fewer than 2 times, replace it with one unknown symbol. In a toy of 6 tokens where only one type repeats, you might keep 2 symbols: that type and unknown. Changing it later silently changes every count and every probability you compute on top.

Map the kept types to ids 0 and 1, and store each sentence as a list of ids. Two sentences of length 6 and 4 need a pad id so the batch is rectangular. You mask pad in the loss. Forget the mask and the model spends updates predicting pad, which might be 2 positions out of 10. If id 0 decodes to the wrong word, every probability you print is about a different token than the one you named.

From a string to ids you can count.

Split and lowercase -> Drop types seen once -> Map survivors to ids -> Mask the pad id

Six tokens, two kept types

1. The toy line has 6 tokens. Only one type occurs at least twice. Every other type is replaced by the unknown symbol.

2. You keep 2 symbols and assign ids 0 and 1. A second sentence of length 4 is padded to length 6, so 2 of 6 positions are pad.

3. Those 2 pad positions must be masked in the loss. If you train on them, 2 of every 6 positions in this batch teach the model to emit pad.

Problem 1. A type occurs once and your cutoff is 2. What do you replace it with?

Answer. The single unknown symbol. Types below 2 occurrences share that symbol, so the vocabulary does not grow a column for a hapax.

Problem 2. A batch row has length 6 and 2 of the positions are pad. What happens if you forget the mask?

Answer. The model is trained to predict pad on those 2 positions. The loss is then partly about the pad id, not about the sentence.

Lecture 2 · 90 minutes

The probability of the next token

A unigram sets a token probability to its count divided by N. On the line the cat sat on the mat, N is 6 and the

occurs 2 times, so its probability is 2/6, or 1/3. A bigram divides by the history count instead. The history the occurs 2 times, once before cat and once before mat, so each of those conditionals is 1/2. At the first position there is no history, and you fall back to the unigram.

A bigram that never saw the pair at test time gets probability 0, and a whole sentence score, the product of the conditionals, becomes 0. Add-1 adds 1 to every bigram count and adds V, the vocabulary size, to every history count. If V is 4 and the history count is 2, a seen conditional that was 1/2 becomes 2/6, and an unseen one becomes 1/6. You should check they sum to 1.

A count is not yet a conditional.

Unigram

Count divided by N

the has probability 1/3

No previous token

Bigram after add-1

Divide by the history

Seen pair becomes 2/6

Unseen pair becomes 1/6

Smooth four next-word values

1. N is 6 and the occurs 2 times, so the unigram probability is 2/6, which is 1/3. The bigram P(cat given the) is 1/2 before smoothing, and P(mat given the) is 1/2.

2. Add-1 with V equal to 4 lifts each seen count from 1 to 2 and the history count from 2 to 6. Each seen conditional becomes 2/6.

3. Each of the 2 unseen next words becomes 1/6. The four probabilities are 2/6, 2/6, 1/6, and 1/6, and they sum to 1. A product that included a raw 0 would have been 0.

Problem 1. On the line the cat sat on the mat, what is the unigram probability of the, and what is P(cat given the) before smoothing?

Answer. The unigram probability is 2/6, or 1/3. P(cat given the) is 1/2, because the history the occurs twice and one of those is followed by cat.

Problem 2. V is 4 and the history count is 2, with two seen next words of count 1. After add-1, what are the two seen probabilities and one unseen probability?

Answer. Each seen probability is 2/6. An unseen probability is 1/6. The denominator is 2 plus 4.

Lecture 3 · 90 minutes

A grammar and one legal tree

You add a grammar when a bag of bigrams is not structure. Three rules suffice: sentence to noun phrase and verb

phrase, noun phrase to determiner and noun, verb phrase to verb and noun phrase. The line the cat sat the mat has 5 tokens. It has a tree if cat and mat are nouns and sat is a verb. You reject the string when no tree exists, even if each bigram probability was above 1/2.

Chart parsing stores spans, not a single guessed tree. A sentence of length 5 has 15 spans. You fill length 1 first, from the part of speech, then longer spans by combining two shorter ones when a rule matches. The cell from 1 to 5 is filled only if some tree covers the sentence. You should be able to name the two child spans that built the verb phrase.

Short spans before the full sentence.

1. Label the 5 tokens

2. Combine spans a rule allows

3. Fill shorter cells first

4. Accept only a full span

Fifteen cells, one verb phrase

1. The sentence the cat sat the mat has length 5. The number of spans is 5 times 6 divided by 2, so 15.

2. Noun phrase covers tokens 1 and 2, the cat. Verb phrase covers tokens 3 through 5, sat the mat. Those two children fill the sentence cell.

3. You can name that split. If the cell from 1 to 5 stays empty, the string is outside this grammar even when every bigram is above 1/2.

Problem 1. A sentence has length 5. How many chart spans are there?

Answer. 15. The count is 5 times 6 divided by 2.

Problem 2. Every bigram probability is above 1/2, but no tree covers the sentence. Do you accept the string?

Answer. No. You accept only when a tree exists under the grammar. Bigram scores do not replace the chart.

Lecture 4 · 90 minutes

A vector that stands in for a word

You compare meanings with vectors, not with raw ids. Let cat be (1, 0) and dog be (0.6, 0.8). Their dot product is 0.6. Both lengths are 1, since 0.6 squared plus 0.8 squared equals 1, so the cosine is also 0.6. Let mat be (0, 1). Its cosine with cat is 0, so you rank dog closer than mat. You still sort by cosine, and you check lengths before trusting a raw dot product.

Roles are not cosines. In the cat sat the mat, mark cat as agent and mat as location. The grammar tree supplies that: in this toy the noun phrase before the verb is the agent, and the other noun phrase is the location. Swap the two phrases and the roles swap, while the cosine between cat and dog stays 0.6. You want both signals. A cosine does not say who sat.

Nearness is not the same fact as a role.

Cosine

cat dot dog is 0.6

Lengths are both 1

mat is cosine 0

Role from the tree

Agent is before the verb

Location is inside the verb phrase

A swap moves roles, not cosine

Dot product 0.6, lengths 1

1. cat is (1, 0) and dog is (0.6, 0.8). The dot product is 1 times 0.6 plus 0 times 0.8, so 0.6.

2. dog's length is the square root of 0.36 plus 0.64, which is the square root of 1, so 1. cat's length is 1. The cosine is 0.6.

3. mat at (0, 1) has cosine 0 with cat. You rank dog closer. Swapping noun phrases in the sentence does not change this 0.6.

Problem 1. cat is (1, 0) and dog is (0.6, 0.8), both of length 1. What is the cosine?

Answer. 0.6. The dot product is 0.6 and you divide by the product of the lengths, which is 1.

Problem 2. You swap the two noun phrases. What happens to the agent role, and what happens to the cosine between cat and dog?

Answer. The agent role moves to the noun phrase that is now before the verb. The cosine stays 0.6. Role and nearness are different signals.

Lecture 5 · 90 minutes

Spans pulled out of a sentence

You score extracted spans by exact match: start, end, and label. If the system returns 4 spans and 3 match, precision is 3/4. If the answer key has 5 spans, recall is 3/5. A span one token too wide counts as a miss, not as half credit. Decide that rule before you read the output. Otherwise the 4th span, the wrong one, becomes a story instead of an error.

You can propose spans from the chart's noun phrases, or from tags. Use 5 tags for person and place: begin and inside for each, plus one shared outside. A span is a begin plus the following inside tags. If your guideline says a lone begin is illegal, do not emit a span for it. Otherwise precision of 3/4 mixes boundary bugs with label bugs, and you cannot tell which of the 4 returned spans failed for which reason.

Propose a span, then demand an exact match.

Tag or take a noun phrase -> Emit begin plus insides -> Demand start, end, and label -> Precision 3/4,

recall 3/5

Three hits out of four returned

1. The system returns 4 spans. Three match the key on start, end, and label. Precision is 3/4.

2. The key has 5 spans, so recall is 3/5. The two missed gold spans are not excused by the three hits.

3. The fourth returned span is one token too wide. It counts as a full miss. You do not give it half credit after looking at it.

Problem 1. The system returns 4 spans and 3 match. The key has 5 spans. What are precision and recall?

Answer. Precision is 3/4 and recall is 3/5. Exact match requires start, end, and label.

Problem 2. A lone begin tag appears and your guideline says that sequence is illegal. Do you emit a span?

Answer. No. A span is a legal begin plus its inside tags. Emitting the lone begin would mix a boundary bug into the precision.

Lecture 6 · 90 minutes

A target sentence and an alignment

You score a translation as a product of token conditionals given the source and the tokens so far. A target of 4 tokens plus a stop decision is 5 factors. If each factor is 1/2, the product is 1/32. Compare candidates by the log sum, 5 times the log of 1/2. A higher score is not a better translation if the reference says otherwise: you still need a separate count of how many words matched.

An alignment links target tokens to source tokens. On a pair of length 3, a monotone alignment has 3 links: positions (1, 1), (2, 2), and (3, 3). Attention replaces a hard link with a distribution over the 3 source positions, and that distribution must sum to 1. If it is flat, each source position gets 1/3, and you have no single link to blame. Do not describe a crisp alignment when the max weight is only 1/3.

Five factors, then a link or a distribution.

1. Score each target token

2. Multiply 4 terms and a stop

3. Product 1/32 if each is 1/2

4. Read attention only if it peaks

Four tokens and a stop

1. The target has 4 tokens. Each conditional is 1/2, and the stop probability is 1/2. That is 5 factors.

2. The product is 1/2 to the fifth, which is 1/32. In logs you add 5 copies of the log of 1/2.

3. A second candidate with product 1/16 beats 1/32. If the reference disagrees, you still count matching words. A flat attention of 1/3, 1/3, 1/3 does not name one source token as the cause.

Problem 1. Four target tokens and a stop decision are each probability 1/2. What is the sentence score?

Answer. 1/32. You multiply five factors of 1/2.

Problem 2. Attention over 3 source positions is 1/3, 1/3, and 1/3. Why should you not narrate a single aligned source word?

Answer. The distribution is flat and already sums to 1. The maximum is 1/3, so the model did not pick one source position.

Lecture 7 · 90 minutes

Precision on sentiment labels

Summarization, dialogue, and sentiment reuse the same pieces with different targets. A summary may be capped at 10 tokens. You still multiply conditionals, and you also require overlap with a source of 40 tokens. A dialogue turn is a generation conditioned on the last 2 user turns plus a retrieved fact. Sentiment is not generation. You classify one sentence into 2 labels, positive or negative, and you report a confusion count rather than a fluent paragraph.

You read a 2 by 2 count, not a single accuracy. On 10 sentences, 6 true positives and 1 true negative give 7 correct, so accuracy is 7/10. The 2 false positives make precision 6/8. The 1 false negative makes recall 6/7. Quote all three. The same counts score a dialogue act or a factual tag on a summary. A string of 10 generated tokens still needs a separate overlap check, because fluency is not the confusion table.

Accuracy hides the two off-diagonal cells.

Accuracy 7/10

6 true positives

1 true negative

7 correct of 10

The rest of the table

2 false positives

Precision 6/8

Recall 6/7

Ten sentences, three numbers

1. The cells are 6 true positives, 2 false positives, 1 false negative, and 1 true negative. They sum to 10.

2. Accuracy is 7/10. Precision for the positive class is 6 divided by 6 plus 2, so 6/8, which is 3/4.

3. Recall is 6 divided by 6 plus 1, so 6/7. You quote all three. A summary capped at 10 tokens still needs an overlap check against its source of 40 tokens. Fluency is not this table.

Problem 1. The cells are 6, 2, 1, and 1 for true positive, false positive, false negative, and true negative. What are accuracy, precision, and recall for the positive class?

Answer. Accuracy is 7/10, precision is 6/8, and recall is 6/7. The 2 false positives are invisible if you quote only accuracy.

Problem 2. A summary is 10 fluent tokens and the source is 40 tokens. Why is the confusion table not enough?

Answer. Sentiment and factual tags can use the table. A generated string still needs an overlap check against the source. Fluency is not a count of true positives.

Lecture 8 · 90 minutes

A pipeline from tokens to a decision

You can move from raw text to a decision you can recompute. You tokenize, drop types seen once, and estimate a bigram. You can test a 5-token sentence against 3 grammar rules and fill 15 chart cells. You can rank two vectors by cosine, pull spans with precision 3/4, and score a 4-token translation as a product of 5 factors of 1/2, which is 1/32. Sentiment ends as a count: accuracy 7/10 is not the same report as precision 6/8.

Put a language-model gate on the extractor. Of 4 returned spans, 3 are correct, and the wrong one has smoothed bigram probability 1/6. Your cutoff is 1/2, so you drop it. Precision moves from 3/4 to 3/3. Recall against 5 gold spans stays 3/5 if that dropped span was not gold. You are done when you can recompute both fractions and say which lecture each one came from.

A decision you can recompute by hand.

Tokens and a bigram

A chart of 15 spans

Cosine, spans, or 1/32

A confusion count

Drop the span whose bigram is 1/6

1. The extractor returns 4 spans and 3 match, so precision is 3/4. The wrong span has smoothed bigram probability 1/6.

2. The cutoff is 1/2. Because 1/6 is below 1/2, you drop that span. Three correct spans remain, so precision is 3/3.

3. The key still has 5 gold spans, and the dropped span was not gold. Recall stays 3/5. You used the bigram lecture and the extraction lecture on the same list.

Problem 1. Four target-token factors and a stop are each 1/2, and a separate sentiment table has accuracy 7/10. Which number is the translation score, and which is not a string probability?

Answer. The translation score is 1/32. Accuracy 7/10 is a count of labels, not a product of token conditionals.

Problem 2. Combine the bigram cutoff with extraction. Of 4 spans, 3 are correct, and the incorrect span has smoothed probability 1/6 under a cutoff of 1/2. The key has 5 gold spans and the dropped span was not gold. What are the new precision and the recall?

Answer. Precision becomes 3/3. Recall stays 3/5. You drop the span because 1/6 is below 1/2, and that span was not one of the gold spans.